Task 5: Transaction-Based Payment Simulation:

Aim:

To simulate an online payment system using SQL transactions, ensuring secure money transfer between user and merchant accounts using COMMIT and ROLLBACK operations.

Algorithm:

1. **Start**
2. **Create Tables**
   - **Create UserAccount table with UserID, Name, Balance**
   - **Create MerchantAccount table with MerchantID, Name, Balance**
3. **Insert Initial Data**
   - **Add sample users and merchants with balances**
4. **Start Transaction**
   - **Begin SQL transaction using BEGIN TRANSACTION**
5. **Check Balance**
   - **Verify if user has sufficient balance**
6. **Deduct Amount**
   - **Subtract payment amount from user account**
7. **Add Amount**
   - **Add the same amount to merchant account**
8. **Commit or Rollback**
   - **If both operations are successful → COMMIT**
   - **If any error occurs or insufficient balance → ROLLBACK**
9. **Display Result**
   - **Show updated balances or error message**
10. **End**

Code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Transaction-Based Payment Simulation</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <style>
        @import
url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&fa
mily=Fira+Code:wght@400;500&display=swap');
        :root {
            --bg-primary: #f8fafc;
            --card-bg: #ffffff;
            --text-main: #1e293b;
        }
        body {
            font-family: 'Inter', sans-serif;
            background-color: var(--bg-primary);
            color: var(--text-main);
        }
        .code-font {
            font-family: 'Fira Code', monospace;
        }
        .transaction-step {
            transition: all 0.3s ease;
        }
        .step-active {
            border-left: 4px solid #3b82f6;
```

```
            background-color: #eff6ff;
        }
        .step-success {
            border-left: 4px solid #10b981;
            background-color: #ecfdf5;
        }
        .step-error {
            border-left: 4px solid #ef4444;
            background-color: #fef2f2;
        }
        @keyframes pulse-subtle {
            0%, 100% { opacity: 1; }
            50% { opacity: 0.7; }
        }
        .processing-pulse {
            animation: pulse-subtle 1.5s infinite;
        }
    </style>
</head>
<body class="p-4 md:p-8">
    <div class="max-w-5xl mx-auto">
        <!-- Header -->
        <header class="mb-8 text-center md:text-left">
            <h1 class="text-3xl font-bold text-slate-900">Payment Transaction
Engine</h1>
            <p class="text-slate-500">Simulating Atomicity (ACID) in Digital
Payments</p>
        </header>
        <div class="grid grid-cols-1 lg:grid-cols-3 gap-8">
```

```
<!-- Left Panel: State & Controls -->
<div class="lg:col-span-1 space-y-6">
<!-- User Account -->
<div class="bg-white p-6 rounded-2xl shadow-sm border border-slate-200">
<div class="flex justify-between items-center mb-4">
<span class="text-xs font-bold uppercase tracking-wider text-slate-400">User Wallet</span>
<div class="p-2 bg-blue-50 rounded-full">
<svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5 text-blue-500" fill="none" viewBox="0 0 24 24" stroke="currentColor">
<path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M16 7a4 4 0 11-8 0 4 4 0 018 0zM12 14a7 7 0 00-7 7h14a7 7 0 00-7-7z" />
</svg>
</div>
</div>
<div class="text-3xl font-bold" id="userBalance">$1000.00</div>
<div class="text-sm text-slate-400 mt-1 italic">Account: #USR-8829</div>
</div>
<!-- Merchant Account -->
<div class="bg-white p-6 rounded-2xl shadow-sm border border-slate-200">
<div class="flex justify-between items-center mb-4">
<span class="text-xs font-bold uppercase tracking-wider text-slate-400">Merchant Store</span>
<div class="p-2 bg-purple-50 rounded-full">
<svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5 text-purple-500" fill="none" viewBox="0 0 24 24" stroke="currentColor">
<path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M16 11V7a4 4 0 00-8 0v4M5 9h14l1 12H4L5 9z" />
```

```
                </svg>
            </div>
        </div>
        <div class="text-3xl font-bold" id="merchantBalance">$5000.00</div>
        <div class="text-sm text-slate-400 mt-1 italic">Account: #MERCH-001</div>
    </div>
    <!-- Payment Form -->
    <div class="bg-slate-900 text-white p-6 rounded-2xl shadow-xl">
        <h3 class="text-lg font-semibold mb-4">Initiate Payment</h3>
        <div class="space-y-4">
            <div>
                <label class="block text-xs font-medium text-slate-400 mb-1">Amount ($)</label>
                <input type="number" id="payAmount" value="250" class="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-2 focus:ring-2 focus:ring-blue-500 outline-none transition-all">
            </div>
            <div class="flex items-center space-x-2">
                <input type="checkbox" id="simulateFailure" class="w-4 h-4 rounded text-blue-600 focus:ring-blue-500">
                <label for="simulateFailure" class="text-sm text-slate-300">Simulate Network Failure</label>
            </div>
            <button onclick="processPayment()" id="payBtn" class="w-full bg-blue-600 hover:bg-blue-500 text-white font-bold py-3 rounded-lg transition-colors flex justify-center items-center space-x-2">
                <span>Execute Transaction</span>
            </button>
        </div>
    </div>
```

```
        </div>
        <!-- Right Panel: Database Transaction Visualizer -->
        <div class="lg:col-span-2 space-y-6">
            <div class="bg-white rounded-2xl shadow-sm border border-slate-200 overflow-hidden flex flex-col h-full">
                <div class="bg-slate-50 border-b border-slate-200 px-6 py-4 flex justify-between items-center">
                    <h3 class="font-bold text-slate-700 flex items-center">
                        <svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5 mr-2 text-slate-400" fill="none" viewBox="0 0 24 24" stroke="currentColor">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 7v10c0 2.21 3.58 4 8 4s8-1.79 8-4V7M4 7c0 2.21 3.58 4 8 4s8-1.79 8-4M4 7c0-2.21 3.58-4 8-4s8 1.79 8 4m0 5c0 2.21-3.58 4-8 4s-8-1.79-8-4" />
                        </svg>
                        DB Transaction Log
                    </h3>
                    <div id="statusBadge" class="hidden px-3 py-1 rounded-full text-xs font-bold uppercase tracking-widest">IDLE</div>
                </div>
                <div id="logContainer" class="p-6 space-y-4 flex-grow min-h-[400px]">
                    <div class="text-center text-slate-400 py-20 italic">
                        Awaiting transaction initiation...
                    </div>
                </div>
                <!-- Database State Snapshot Footer -->
                <div class="bg-slate-900 p-4 code-font text-xs text-blue-300 flex justify-around">
                    <div>DB_STATUS: <span id="dbStatusText" class="text-green-400 underline">STABLE</span></div>
                    <div>ACTIVE_LOCKS: <span id="lockStatus" class="text-slate-400">NONE</span></div>
```

```
                </div>
            </div>
        </div>
    </div>
</div>
<!-- UI Feedback Toast -->
<div id="toast" class="fixed bottom-8 left-1/2 -translate-x-1/2 px-6 py-3 rounded-xl shadow-2xl transition-all opacity-0 pointer-events-none z-50"></div>
<script>
    let balances = {
        user: 1000.00,
        merchant: 5000.00
    };
    const logs = document.getElementById('logContainer');
    const statusBadge = document.getElementById('statusBadge');
    const dbStatusText = document.getElementById('dbStatusText');
    const lockStatus = document.getElementById('lockStatus');
    function updateUI() {
        document.getElementById('userBalance').innerText = `$${balances.user.toFixed(2)}`;
        document.getElementById('merchantBalance').innerText = `$${balances.merchant.toFixed(2)}`;
    }
    function createStep(text, code, type = 'pending') {
        const div = document.createElement('div')
     div.className = `transaction-step p-4 rounded-xl border border-slate-100 ${type === 'active' ? 'step-active' : ''}`;
        div.innerHTML = `
            <div class="flex justify-between items-start">
                <div class="text-sm font-semibold text-slate-700">${text}</div>
```

```
                <div class="text-[10px] code-font bg-slate-100 px-2 py-0.5 rounded text-slate-500">${code}</div>
            </div>
        `;
        return div;
    }
    function showToast(msg, isError = false) {
        const t = document.getElementById('toast');
        t.innerText = msg;
        t.className = `fixed bottom-8 left-1/2 -translate-x-1/2 px-6 py-3 rounded-xl shadow-2xl transition-all z-50 ${isError ? 'bg-red-600 text-white' : 'bg-green-600 text-white'}`;
        t.style.opacity = "1";
        setTimeout(() => t.style.opacity = "0", 3000);
    }
    async function processPayment() {
        const amount = parseFloat(document.getElementById('payAmount').value);
        const failSim = document.getElementById('simulateFailure').checked;
        const btn = document.getElementById('payBtn');
        if (isNaN(amount) || amount <= 0) {
            showToast("Please enter a valid amount", true);
            return;
        }
        // Initialization
        btn.disabled = true;
        btn.classList.add('opacity-50', 'cursor-not-allowed');
        logs.innerHTML = '';
        statusBadge.classList.remove('hidden', 'bg-green-100', 'text-green-700', 'bg-red-100', 'text-red-700');
        statusBadge.classList.add('bg-blue-100', 'text-blue-700');
```

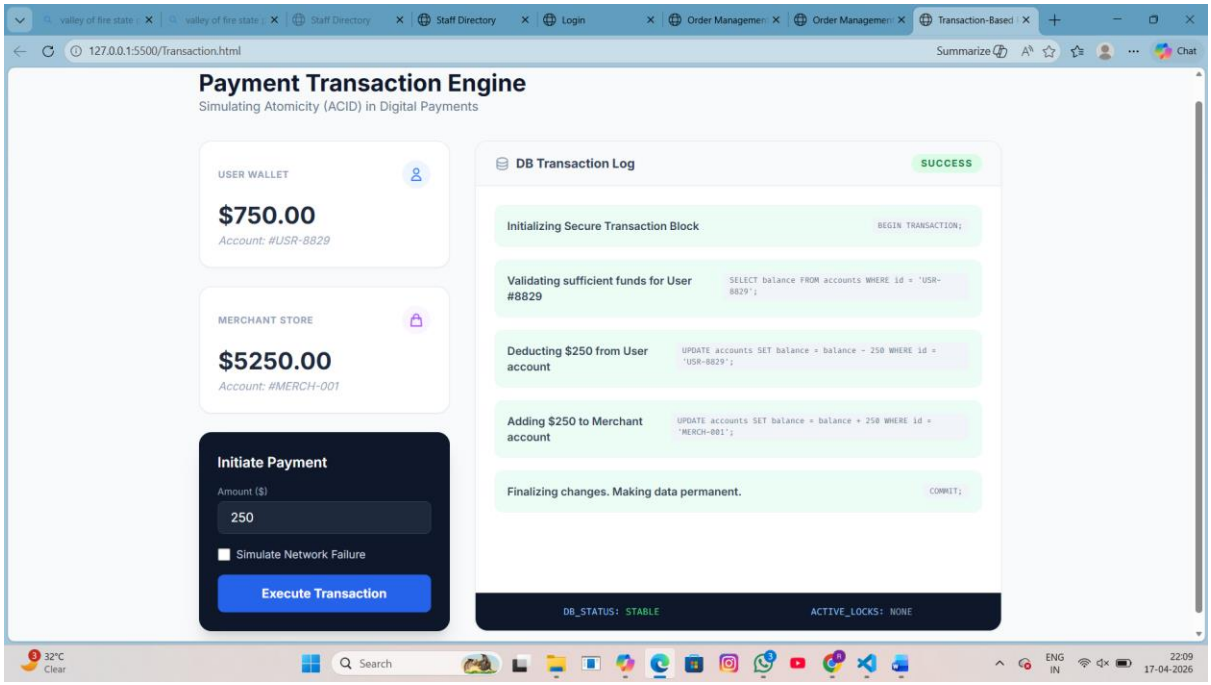
```
statusBadge.innerText = 'PROCESSING';
dbStatusText.innerText = 'IN_TRANSACTION';
dbStatusText.className = 'text-yellow-400';
lockStatus.innerText = 'ROW_EXCL_LOCK';
try {
    // Step 1: BEGIN
    const step1 = createStep("Initializing Secure Transaction Block",
"BEGIN TRANSACTION;", "active");
    logs.appendChild(step1);
    await new Promise(r => setTimeout(r, 800));
    step1.classList.replace('step-active', 'step-success');
    // Step 2: Check Balance
    const step2 = createStep("Validating sufficient funds for User #8829",
`SELECT balance FROM accounts WHERE id = 'USR-8829';`, "active");
    logs.appendChild(step2);
    await new Promise(r => setTimeout(r, 1000));
    if (balances.user < amount) {
        throw new Error("INSUFFICIENT_FUNDS");
    }
    step2.classList.replace('step-active', 'step-success');
    // Step 3: Deduct User
  const step3 = createStep(`Deducting $${amount} from User account`,
`UPDATE accounts SET balance = balance - ${amount} WHERE id = 'USR-
8829';`, "active");
    logs.appendChild(step3);
    await new Promise(r => setTimeout(r, 1200));
    // Temporary local mutation (representing uncommitted DB state)
    const originalUserBalance = balances.user;
    balances.user -= amount;
    updateUI();
```

```
            step3.classList.replace('step-active', 'step-success');
            // SIMULATED CRASH POINT
            if (failSim) {
                throw new Error("NETWORK_TIMEOUT_DURING_UPDATE");
            }
            // Step 4: Add Merchant
            const step4 = createStep(`Adding $${amount} to Merchant account`,
`UPDATE accounts SET balance = balance + ${amount} WHERE id = 'MERCH-
001';`, "active");
            logs.appendChild(step4);
            await new Promise(r => setTimeout(r, 1200));
            balances.merchant += amount;
            updateUI();
            step4.classList.replace('step-active', 'step-success');
            // Step 5: COMMIT
            const step5 = createStep("Finalizing changes. Making data
permanent.", "COMMIT;", "active");
            logs.appendChild(step5);
            await new Promise(r => setTimeout(r, 800));
            step5.classList.replace('step-active', 'step-success');
            statusBadge.classList.replace('bg-blue-100', 'bg-green-100');
            statusBadge.classList.replace('text-blue-700', 'text-green-700');
            statusBadge.innerText = 'SUCCESS';
            showToast("Payment Successful!");
        } catch (error) {
            // Handle Rollback
            const errorStep = document.createElement('div');
            errorStep.className = "p-4 rounded-xl border border-red-200 bg-red-
50 text-red-700 mb-4";
            errorStep.innerHTML = `<strong>Error:</strong> ${error.message}`;
```

```
            logs.appendChild(errorStep);
            const rollbackStep = createStep("Failure detected. Reverting all
changes to previous state.", "ROLLBACK;", "active");
            rollbackStep.classList.add('step-error');
            logs.appendChild(rollbackStep);
            await new Promise(r => setTimeout(r, 1500));
            // Revert balances (The Rollback Magic)
            if (error.message !== "INSUFFICIENT_FUNDS") {
                // Logic to reset to state before transaction started
                // In a real DB, this happens automatically
                location.reload(); // Hard reset for simulation clarity if needed, or:
            }
            statusBadge.classList.replace('bg-blue-100', 'bg-red-100');
            statusBadge.classList.replace('text-blue-700', 'text-red-700');
            statusBadge.innerText = 'FAILED';
            showToast(`Transaction Failed: ${error.message}`, true);
            // Manual rollback of UI state if we didn't reload
            balances.user = 1000;
            balances.merchant = 5000;
            updateUI();
        } finally {
            btn.disabled = false;
            btn.classList.remove('opacity-50', 'cursor-not-allowed');
            dbStatusText.innerText = 'STABLE';
            dbStatusText.className = 'text-green-400';
            lockStatus.innerText = 'NONE';
        }
    }
    // Initial UI Load
```

```
        updateUI();
    </script>
</body>
</html>
```

Output:



Result

A transaction-based payment simulation system was successfully implemented using SQL transaction control commands. The system demonstrates secure money transfer between a user account and a merchant account by deducting the amount from the user and crediting it to the merchan